Drawing a World Map with D3

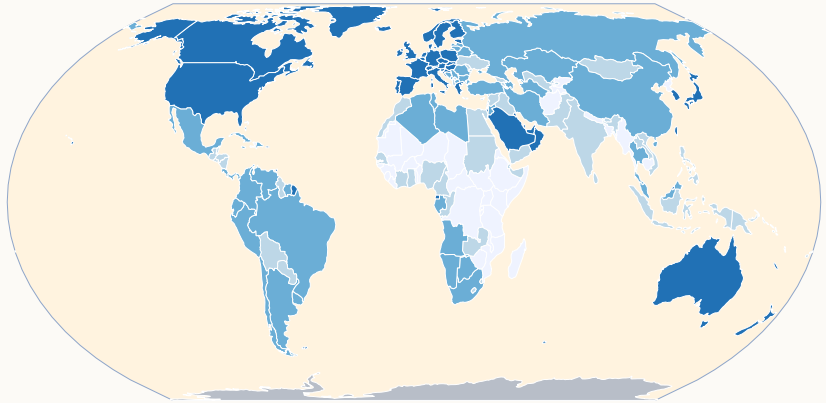
HTML, JavaScript, and SVG

Information Visualization · Lecture 17 · Lab

Today’s program grows one step at a time

- **Review D3:** select elements, use scales, and bind data.
- **Draw a world map:** load GeoJSON, set a projection, and generate SVG paths.
- **Color by income:** build a colormap and color each country.
- **Extend with AI:** add a legend and mouse interaction; inspect and test the result.

Appendix: legend and event walkthroughs, optional CSV joins, and complete reference code.



Open Worldmap.html (outline map) ↗ · Download all examples ↓

Natural Earth v5.1.2 · public domain · legacy income groups; observation year unspecified

Run the lab with VS Code Live Server

```
Lecture17_examples/
  D3_intro.html
  Worldmap.html
  outline_map.js
  income_map.js
  data/  lib/  style.css
```

Download examples ZIP ↓, extract it, and open the entire `Lecture17_examples` folder in VS Code.

D3 basics are in D3_intro.html. Save changes to reload. Use http://, not file://.

1. Install or enable **Live Server** by Ritwick Dey.
2. Right-click `D3_intro.html` → **Open with Live Server**.
3. For maps, open `Worldmap.html` the same way.
4. Change its map script `src` to `outline_map.js`, then `income_map.js`. Load **one map script at a time**.

D3 connects JavaScript data to SVG

D3 is a JavaScript library for transforming data and creating web visualizations. Its functions select elements, calculate drawing values, and modify the DOM.

- **HTML** provides the page and an SVG container.
- **JavaScript** holds records and runs functions.
- **D3** helps create and configure SVG elements.

```
const svg =
d3.select("#chart");
svg.append("rect")
  .attr("width", 160)
  .attr("height", 30);
```

The browser displays the resulting SVG elements.

D3 documentation

The library loads before the chart program

```
<!-- D3_intro.html: inside <head> -->
<script src="lib/d3.v7.9.0.min.js"></script>
<!-- inside <body> -->
<svg id="chart" viewBox="0 0 520 240"></svg>
<!-- after </body>: the supplied inline script -->
<script>
  // D3 chart code is written here.
</script>
```

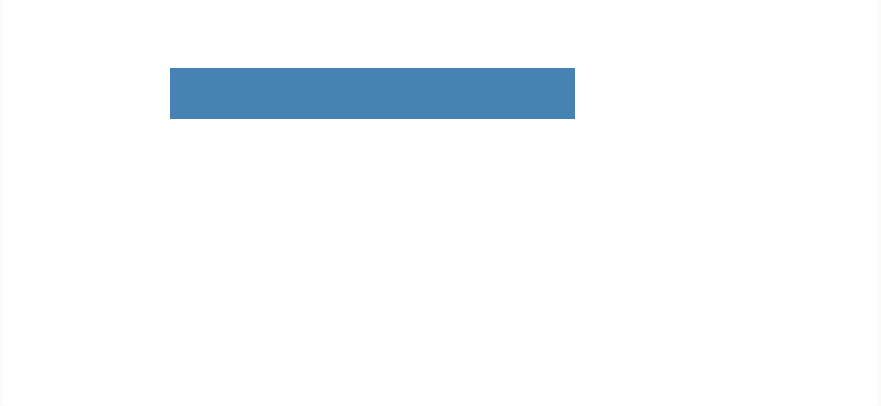
The library loads first. The inline script runs after the SVG exists. Edit that script in `D3_intro.html`; there is no separate chart.js file.

`viewBox` defines the SVG drawing coordinates: 520 units wide and 240 units high.

select finds an element; append creates a child

A **D3 selection** wraps DOM elements so that D3 methods can act on them.

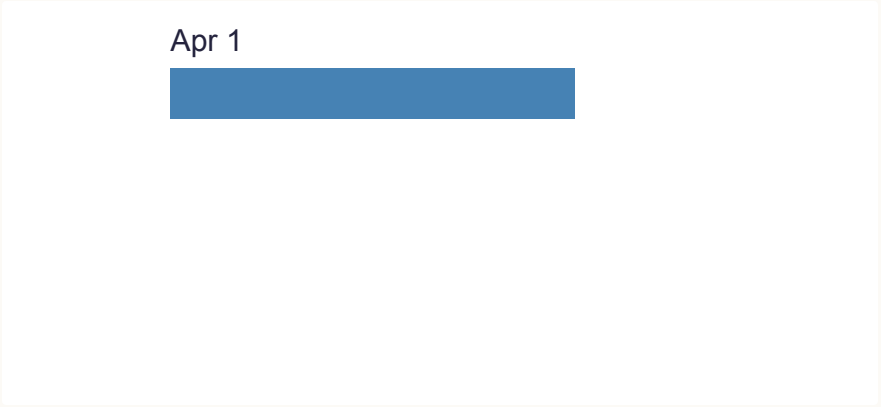
```
const svg = d3.select("#chart");
const bar = svg.append("rect");
bar.attr("x", 100)
  .attr("y", 40)
  .attr("width", 240)
  .attr("height", 30)
  .attr("fill", "steelblue");
```



Edit the inline script in D3_intro.html ↗ · Replace its chart code with this snippet; reload. D3 selections: append and attr

text sets the content of a text element

```
svg.append("text")
  .attr("x", 100)
  .attr("y", 30)
  .text("Apr 1");
```



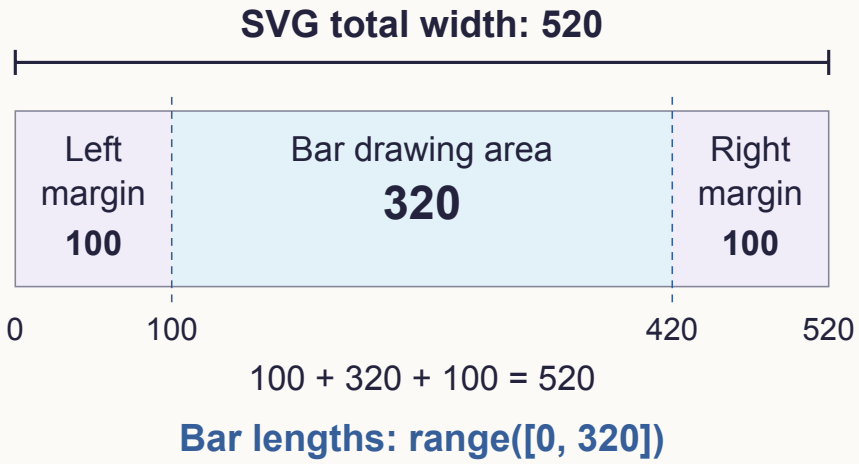
`attr()` sets an attribute such as position. `text()` sets the element's text content. The text's `y` coordinate locates its baseline.

Open D3_intro.html ↗ · Change the label and save.

A scale maps an input domain to an output range

A **scale** is a function that converts data values into drawing values. A linear scale interpolates proportionally between its endpoints.

```
const x = d3.scaleLinear()
  .domain([0, 400])
  .range([0, 320]);
console.log(x(200)); // 160
```



Domain: 0–400 trips. **Range:** 0–320 SVG units of bar length.

Open D3_intro.html ↗ · D3 linear scales

forEach repeats the drawing instructions

```
daily.forEach((d, i) => {
  svg.append("rect")
    .attr("x", 100)
    .attr("y", 40 + i * 60)
    .attr("width", x(d.rides))
    .attr("height", 30)
    .attr("fill", "steelblue");
});
```

Open D3_intro.html ↗

Citi Bike · Jersey City · April 1–3, 2020

`daily` contains three objects:

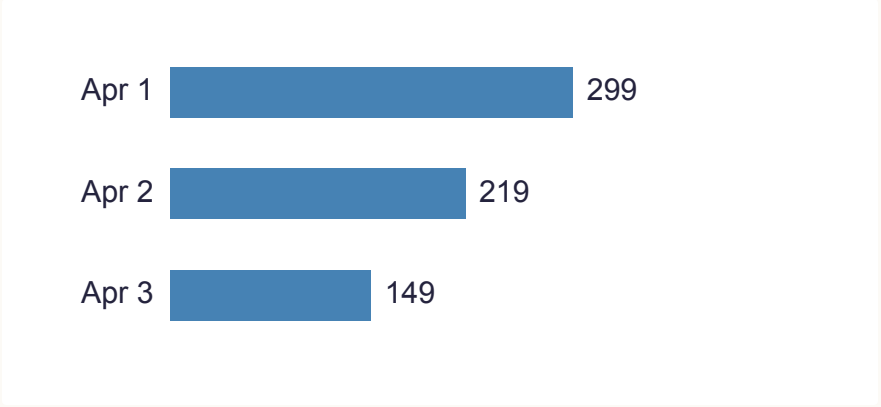
```
[
  {date: "Apr 1", rides: 299},
  {date: "Apr 2", rides: 219},
  {date: "Apr 3", rides: 149}
]
```

Here `d` is one record and `i` is its array index.

Data binding associates records with elements

Data binding associates a record with a DOM element so later callbacks can use that record.

```
svg.selectAll("rect")
  .data(daily)
  .enter().append("rect")
  .attr("x", 100)
  .attr("y", (d, i) => 40 + i * 60)
  .attr("width", d => x(d.rides))
  .attr("height", 30)
  .attr("fill", "steelblue");
```



Open D3_intro.html ↗ · Compare its output with `forEach`.

D3 data binding

data matches records without creating elements

```
const bars = svg.selectAll("rect").data(daily);
```



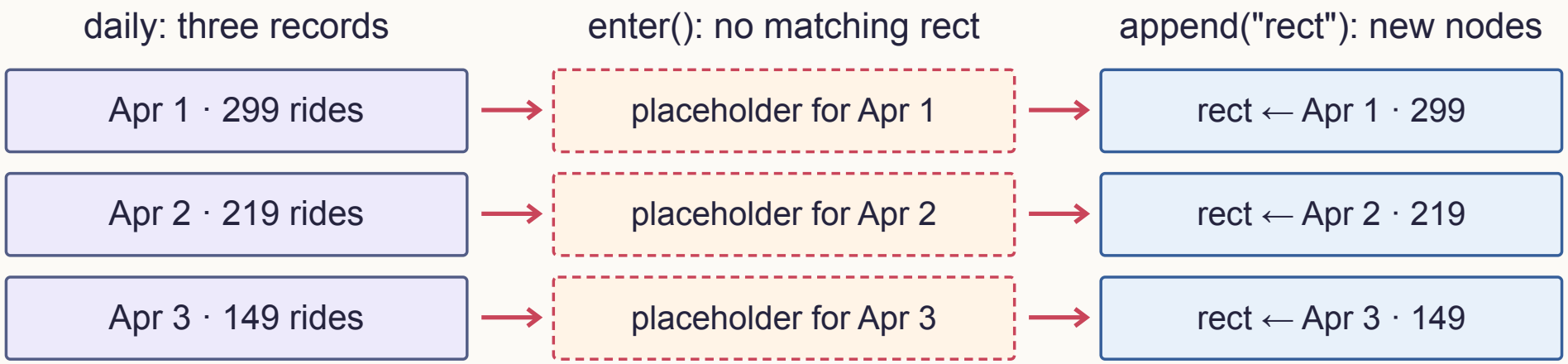
SVG: 0 rectangles before → 0 rectangles after data(daily)

Starting with an empty SVG, `selectAll("rect")` finds zero rectangles. After `data(daily)`, there are still **zero rectangles**.

D3 has identified three records that need new elements.

enter identifies the records that need elements

```
bars.enter().append("rect");
```



`enter()` returns placeholders for unmatched records. `append("rect")` creates a rectangle for each placeholder and retains its record.

The rectangles still need position and size before they form readable bars.

Attribute callbacks receive the bound record

```
svg.selectAll("rect")
  .attr("x", 100)
  .attr("y", (d, i) => 40 + i * 60)
  .attr("width", d => x(d.rides))
  .attr("height", 30);
```

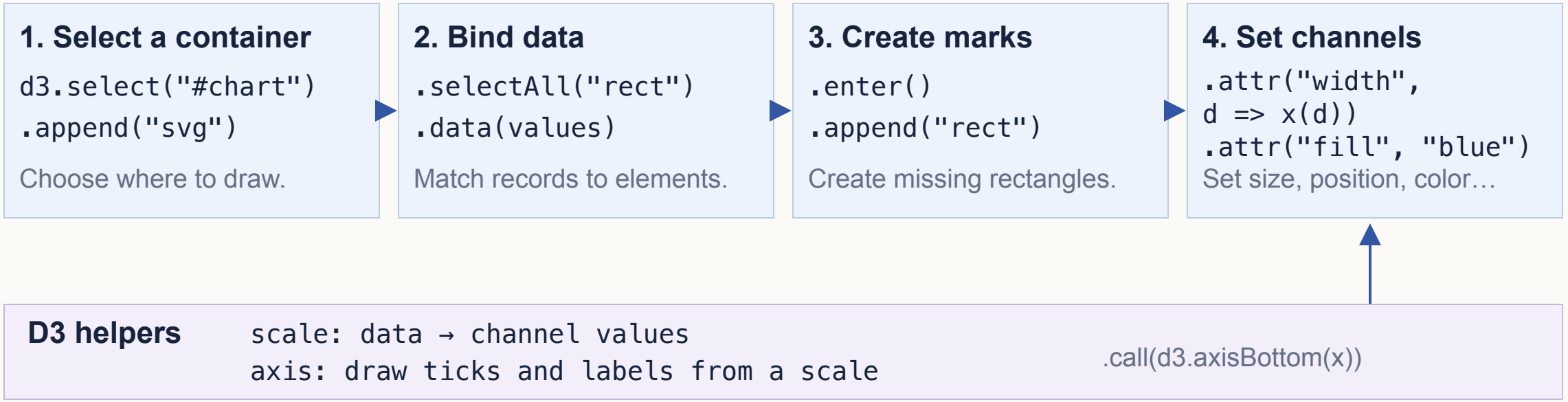
Record	i	Width
Apr 1 · 299	0	239.2
Apr 2 · 219	1	175.2
Apr 3 · 149	2	119.2

A constant such as `100` applies to every selected rectangle. A callback computes a value separately for each rectangle.

Predict: what changes if the second record becomes 300 trips?

The big picture: data becomes marks and channels

D3 builds a visualization by selecting a drawing container, binding records to elements, creating marks, and setting their visual attributes.



Start with `d3.select("body")` or `d3.select("#chart")` for a container such as `<div id="chart">`. If the SVG already exists, select it directly.

First render on an empty SVG. `data` binds records; `enter().append()` creates elements. Add interaction later with `.on(...)`.

D3 built-in functions

Call library functions on `d3`; call **selection methods** on selected DOM elements.

Task	Common functions / methods	What they do
1. DOM operations	`d3.select()`, `d3.selectAll()` `selection.append()`	Find elements and create children.
2. Load and bind data	`d3.csv()`, `d3.json()` `selection.data()`, `.enter()`	Load files, bind records and identify records needing elements.
3. Marks and channels	`selection.append()`, `.attr()` `d3.line()`, `d3.geoPath()`	Create marks, set attributes and generate path commands.
4. Interaction	`selection.on()`	Attach event listeners, such as click or mouseenter.
5. Helpers	`d3.scaleLinear()`, `d3.axisBottom()` `d3.min()`, `d3.max()`	Map values, generate axes and summarize data.

Examples use D3 v7 API names. D3 API reference · These are teaching groups; a function can support more than one task.

Draw a world map using D3

The big picture: country coordinates become SVG paths

1. Array of country features

```
world.features
[
  { ... },
  { ... }, ...
]
```

Indonesia

`geometry.coordinates:`
`[141.0, −2.6], ...`
longitude, latitude (degrees)

2. Convert the coordinates

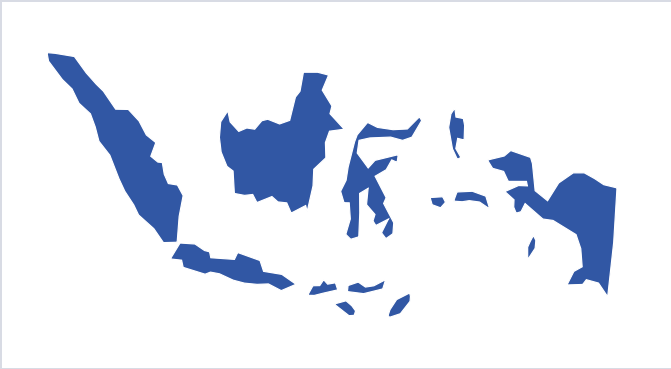
Projection
`geoEqualEarth()`

`[x, y] = [285.0, 76.4]`

`geoPath(projection)`

3. SVG draws the outline

`<path d="M…L…Z" />`



One feature → one <path>

Coordinate excerpt rounded for display. D3 repeats the process for every feature; one path can contain multiple islands.

```
svg.selectAll("path.country").data(world.features)
  .enter().append("path").attr("class", "country")
  .attr("d", d => path(d));
```

Open Worldmap.html (outline map) ↗ · Country geometry: Natural Earth v5.1.2 · `path = d3.geoPath(projection)` · SVG's `d` attribute stores drawing commands.

Step 1 · Load geographic data with d3.json

`d3.json(url)` fetches a JSON document and parses its text into a JavaScript value. It returns a **Promise** representing the eventual result.

```
d3.json("data/world-income.geojson")
  .then(world => {
    console.log(world);
  })
  .catch(error => console.error(error));
```

The URL is relative to the HTML page. Run the example through Live Server so the browser can request the local data file.

Open Worldmap.html (outline map) ↗

D3 json

then receives the data after loading succeeds

Loading is asynchronous. The callback passed to `then` runs when the Promise resolves with the parsed data.

```
d3.json("data/world-income.geojson")
  .then(world => {
    console.log(world.features.length); // 177
    drawMap(world);
  })
  .catch(showError);
```

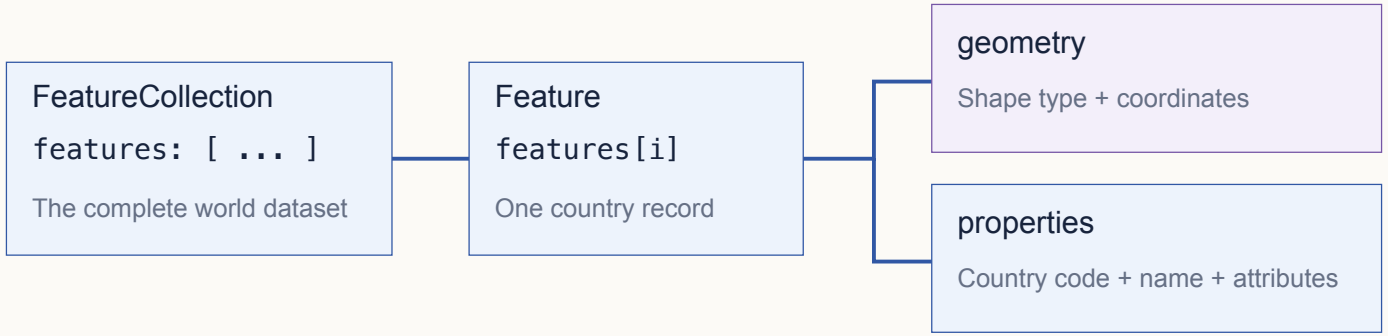
`world` is our parameter name. `drawMap` and `showError` are functions we write. Code that needs the loaded data belongs inside this callback.

D3 json

The GeoJSON Format

GeoJSON represents geographic data as JSON objects made of **key–value pairs**. The `"type"` key identifies the object's role and determines which other keys it contains.

Common values of `"type"`	Other keys	What the object represents
`"FeatureCollection"`	`features`	An array of Features
`"Feature"`	`geometry`, `properties`	A shape with descriptive attributes
`"Point"`, `"LineString"`, `"Polygon"`, `"MultiPolygon"`	`coordinates`	Geometry: a shape and its positions



GeoJSON specification · Geometry types are values of type; `"Geometry"` is not itself a type value.

A GeoJSON example connects keys to their values

GeoJSON stores geographic data as JSON **key–value pairs**. Follow the colors from the outer collection to its Feature and nested geometry.

```
{
  "type": "FeatureCollection",
  "features": [{
    "type": "Feature",
    "properties": {"name": "Site A"},
    "geometry": {
      "type": "Point",
      "coordinates": [125.6, 10.1]
    }
  }]
}
```

1 · FeatureCollection

`features` holds an array of Features.

2 · Feature

`properties` holds descriptive attributes.

`geometry` holds one geometry object.

3 · geometry → Point

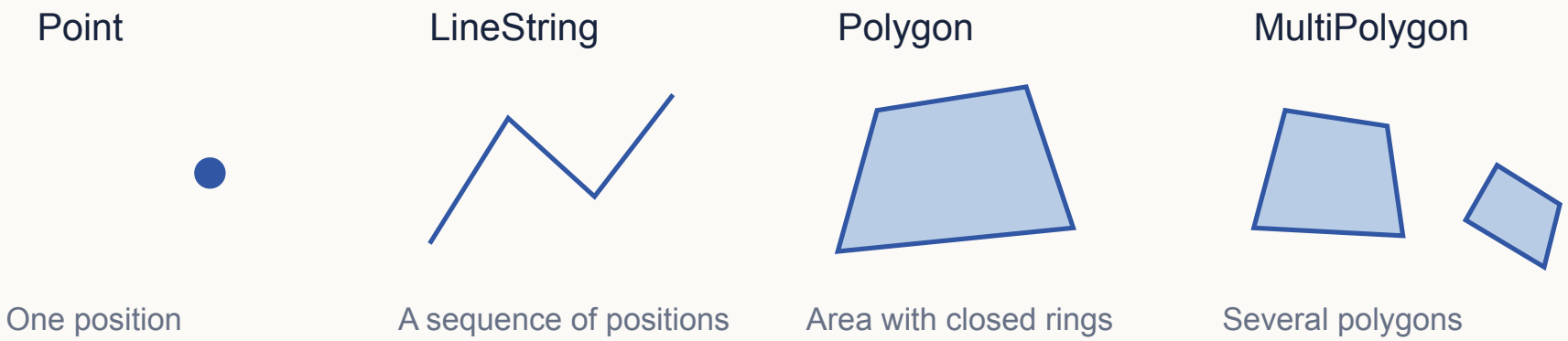
`coordinates` holds `[longitude, latitude]`.

Other geometry type values include `LineString`, `Polygon` and `MultiPolygon`.

Hierarchy: FeatureCollection → features[] → Feature → geometry + properties. GeoJSON specification · Illustrative point.

Geometry type tells us how to read coordinates

A **geometry object** specifies its shape with `type`. Most types store positions in `coordinates`, with nesting determined by the type.



GeoJSON has seven geometry types: these four, plus `MultiPoint`, `MultiLineString`, and `GeometryCollection`.

`GeometryCollection` uses a `geometries` array instead of `coordinates`.

GeoJSON specification

Polygon coordinates are arrays of closed rings

```geojson
{
  "type": "Polygon",
  "coordinates": [
    [ [0, 0], [2, 0], [2, 1],
      [0, 1], [0, 0] ]
  ]
}
```

A position is `[longitude, latitude]` in degrees. Longitude comes first.

A ring is an array of positions. Its final position repeats its first position.

The first ring is the exterior boundary. Later rings, if present, describe holes.

MultiPolygon adds one level: polygons → rings → positions → longitude / latitude.

GeoJSON specification

A GeoJSON feature combines geometry and properties

```
{
  "type": "Feature",
  "properties": {
    "code": "CHN",
    "name": "China"
  },
  "geometry": {
    "type": "MultiPolygon",
    "coordinates": [ ... ]
  }
}
```

GeoJSON specification · structural excerpt; coordinates abbreviated

A **Feature** attaches descriptive properties to one geometry object.

- `properties`: identifiers and descriptive attributes.
- `geometry`: shape type and coordinates.
- A `MultiPolygon` feature can contain several separate pieces.

FeatureCollection provides the array for data binding

```
{
  "type": "FeatureCollection",
  "features": [
    { "type": "Feature", ... },
    { "type": "Feature", ... }
  ]
}
```

`world` is the complete collection object.

`world.features` is an array of country records.

`world.features[0]` is one Feature, with its own geometry and properties.

Later, `.data(world.features)` will supply one record for each country path.

GeoJSON specification

TopoJSON stores shared boundaries once

TopoJSON is a JSON format that encodes topology: connected line segments called **arcs** can be referenced by more than one geometry.



Natural Earth v5.1.2 · simplified country boundaries

Shared Portugal–Spain border

GeoJSON

Both countries include this border's coordinates.

TopoJSON

Both countries reference the same stored arc.

A shared border is stored once, reducing repeated coordinates. The top-level type is `Topology`.

TopoJSON specification · Geometry: Natural Earth v5.1.2 · public domain

The TopoJSON Format

A TopoJSON document is a JSON object with `"type": "Topology"` and the following fields.

```
{
  "type": "Topology",
  "objects": {
    "countries": {
      "type": "GeometryCollection",
      "geometries": [ ... ]
    } },
  "arcs": [ ... ],
  "transform": {
    "scale": [0.01, 0.01],
    "translate": [-180, -90]
  }
}
```

objects

A dictionary of named geometry objects. Names such as `countries`, `land` or `Portugal` are chosen by the file's author.

arcs

Shared coordinate sequences. Lines and polygons refer to their indices.

transform — optional

Scale and translation used to decode quantized coordinates. This is not a map projection.

Structural example; arrays abbreviated and transform values illustrative. TopoJSON specification

Convert TopoJSON to GeoJSON with topojson.feature

Load `topojson-client` after D3 and before running the conversion snippet. It exposes the global object `topojson`.

```
<!-- Add after D3 in Worldmap.html -->
<script src="lib/topojson-client.v3.1.0.min.js"></script>
```

```
d3.json("data/portugal-spain.topojson")
.then(topology => {
  const country = topojson.feature(
    topology,                    // 1
    topology.objects.Portugal // 2
  );
  console.log(country);
});
```

1 · topology

The complete loaded TopoJSON object, including its shared arcs.

2 · object

The geometry object to convert. Select an existing name from `topology.objects`.

View portugal-spain.topojson ↗ · GeometryCollection → FeatureCollection; Polygon → Feature. Run this snippet in Console after adding the local library. topojson-client API

Inspect one feature before drawing the map

```
console.log(world.type);
console.log(world.features.length);
console.log(world.features[0].properties);
console.log(world.features[0].geometry.type);
```

Expected output

```
FeatureCollection
177
{code: "FJI", name: "Fiji"}
MultiPolygon
```

Natural Earth v5.1.2 · public domain · legacy income groups; observation year unspecified

Your check

Find `CHN` in the features array. Which object contains its name? Which object contains its outline?

Step 2 · Project longitude and latitude to SVG coordinates

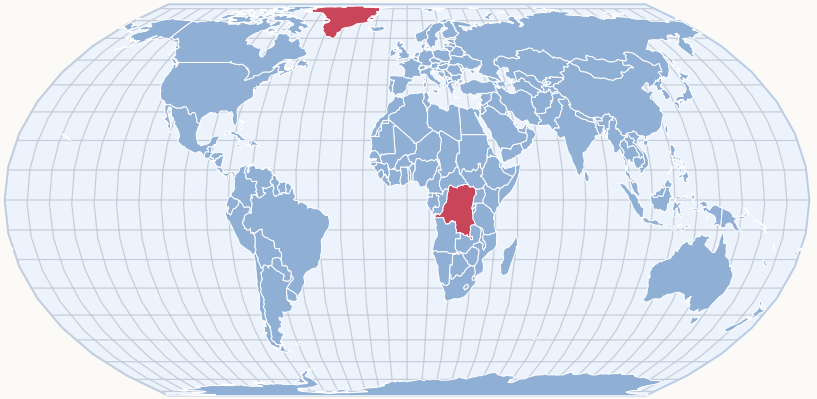
A **map projection** is a mathematical transformation from geographic positions on Earth to positions on a flat map.

A globe view



→

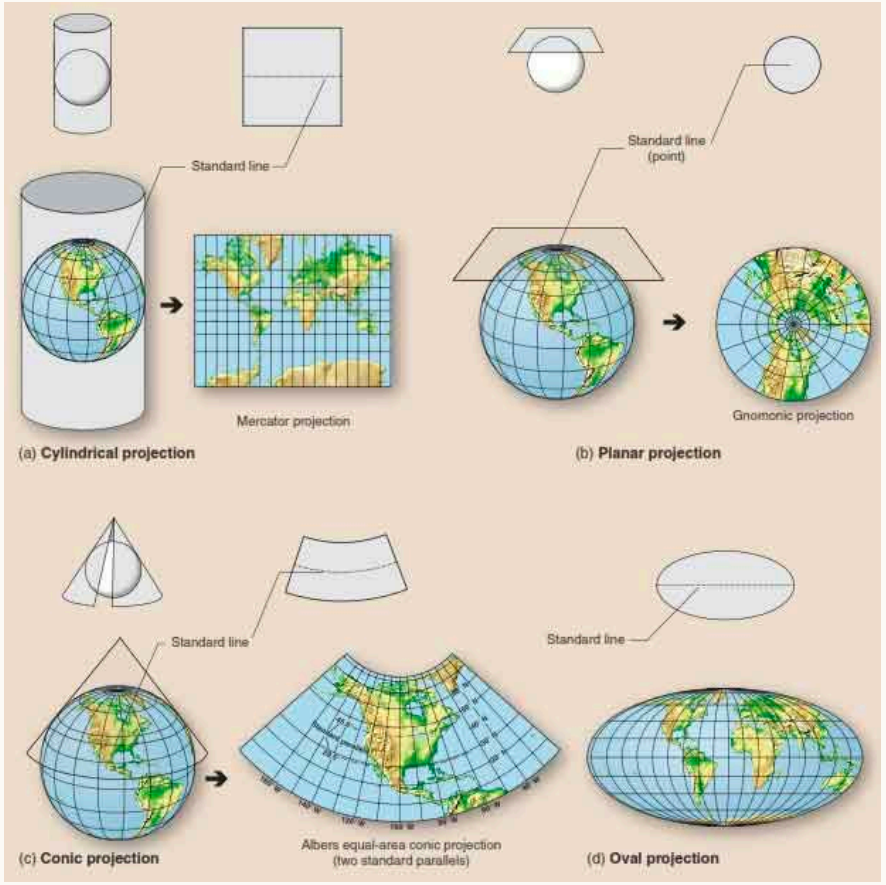
A flat world map



Flattening a curved surface introduces distortion. A projection cannot preserve area, shape, distance and direction everywhere at once.

Geometry: Natural Earth v5.1.2 · Rendered with D3 geographic projections.

Choose map projections for the region and task



Cylindrical

Often suited to equatorial regions. In Mercator, area distortion increases toward the poles.

Planar / azimuthal

Useful for views centered on a location, such as a pole. Distortion depends on distance from the center and the formula.

Conic

Often suited to mid-latitude regions. Standard parallels help control distortion across the region.

Choose what to preserve—area, local shape, distance or direction—and configure the projection for the region.

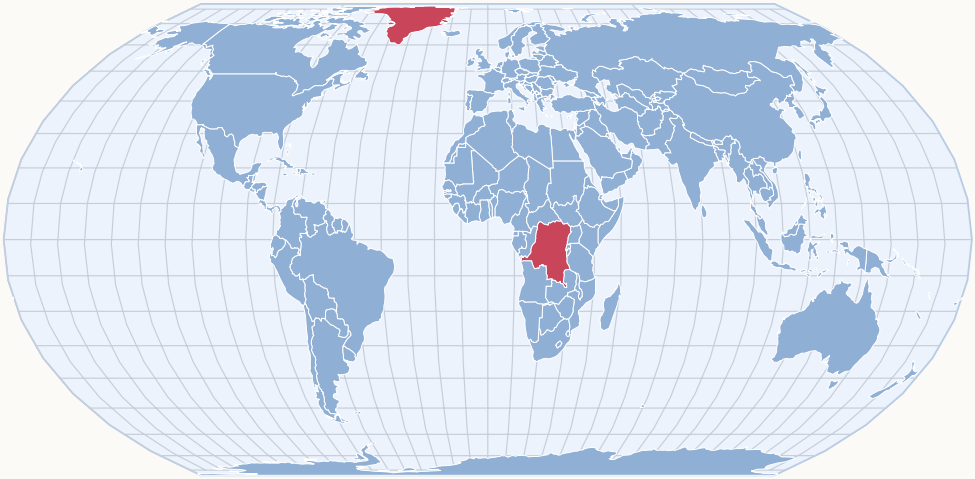
Mercator enlarges areas toward the poles

Mercator is a conformal cylindrical projection: it preserves local angles, but exaggerates area increasingly at high latitudes.

Mercator



Equal Earth



Compare the red shapes: Greenland and DR Congo. **Equal Earth preserves relative areas**; Mercator makes Greenland appear much larger relative to equatorial countries.

Geometry: Natural Earth v5.1.2 · Rendered with D3 geographic projections.

D3 provides projection functions ready to use

A D3 projection is a function: it accepts `[longitude, latitude]` in degrees and returns drawing coordinates `[x, y]`.

D3 constructor	Use in this course
`d3.geoEqualEarth()`	Equal-area world map; our lab's default.
`d3.geoMercator()`	Compare local shape preservation and area distortion.
`d3.geoOrthographic()`	A globe-like view of one hemisphere.

```
const projection = d3.geoEqualEarth();
const point = projection([116.4, 39.9]); // [x, y]
```

These functions are in the supplied D3 library. Additional projections are available in the separate `d3-geo-projection` extension.

D3 projections · Extended projection gallery

From GeoJSON to a world map

```
const svg = d3.select("#map");
d3.json("data/world-income.geojson")
.then(world => {
  const projection = d3.geoEqualEarth()
    .fitExtent([[20, 20], [980, 470]], world);
  const path = d3.geoPath(projection);
  svg.selectAll("path.country")
    .data(world.features)
    .enter().append("path")
    .attr("class", "country")
    .attr("d", d => path(d))
    .attr("fill", "steelblue")
    .attr("stroke", "white")
    .attr("stroke-width", 0.5);
}).catch(error => console.error(error));
```

1. Load the data

`then` receives the GeoJSON collection as `world`.

2. Project and generate paths

Equal Earth preserves area. `fitExtent` fits the map between the two corners. `geoPath(projection)` converts geometry to SVG path commands.

3. Bind and draw

Bind `world.features`, append one `path` per Feature, then set its shape, fill and boundary.

HTML needs `<svg id="map" viewBox="0 0 1000 650"></svg>` and D3 loaded first. Map extent: [20, 20]–[980, 470]. Open Worldmap.html (outline map) ↗

geoPath turns geographic shapes into SVG paths

An SVG `<path>` draws a shape from commands in its `d` attribute. A single path can contain multiple lines, curves and closed outlines.

```
<path d="M20,20 L120,20 L80,80 Z"
      fill="steelblue" />
```



`M` : move · `L` : line · `Z` : close.

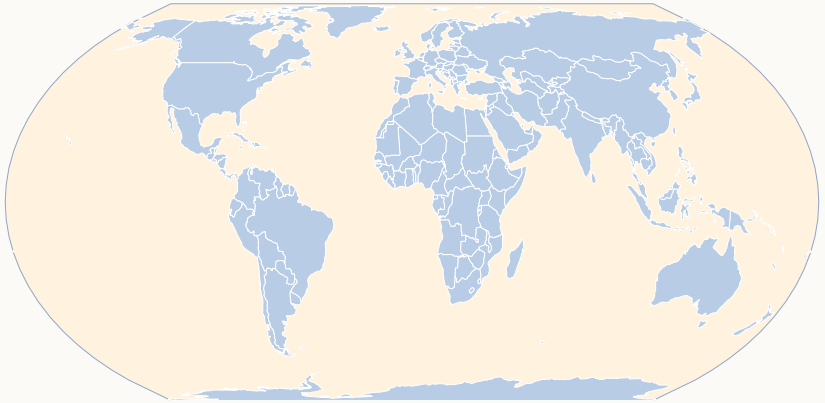
```
const path = d3.geoPath(projection);
const country = world.features[0];
svg.append("path").attr("d", path(country));
```

`geoPath` applies the projection to each country’s geometry and generates its SVG path commands. One path per Feature produces the world map.

D3 geographic paths · `projection` maps longitude/latitude to x/y; `path(feature)` returns the `d` string.

Draw the background, then draw the countries

```
const svg = d3.select("#map");
// 1. Load the data
d3.json("data/world-income.geojson")
.then(world => {
  // 2. Set projection and geoPath
  const projection = d3.geoEqualEarth()
    .fitExtent([[20, 20], [980, 470]], world);
  const path = d3.geoPath(projection);
  // 3. Draw the Sphere background
  svg.append("path")
    .attr("d", path({type: "Sphere"}))
    .attr("fill", "#fff3df")
    .attr("stroke", "#8fa6ca");
  // 4. Draw all countries
  svg.selectAll("path.country")
    .data(world.features).enter().append("path")
    .attr("class", "country")
    .attr("d", d => path(d))
    .attr("fill", "#b8cce5")
    .attr("stroke", "white")
    .attr("stroke-width", 0.5);
}).catch(error => console.error(error));
```



A background for the whole globe

`{type: "Sphere"}` represents the entire globe in D3's extended GeoJSON format.

`path()` generates the projected outline. Append this path **first**, then draw the countries on top.

Open Worldmap.html (outline map) ↗ · D3 geographic streams: Sphere · Geometry: Natural Earth v5.1.2 · public domain

Income group is an ordinal attribute with four levels

An **ordinal attribute** has categories with a meaningful order, but the gaps between categories are not measured as equal numeric intervals.

Actual properties from the lab data

`world-income.geojson` · China feature

```
"properties": {
  "code": "CHN",
  "name": "China",
  "income_group": "Upper middle income"
}
```

Read the category with `d.properties.income_group`.

Four levels, from low to high

1. Low income
2. Lower middle income
3. Upper middle income
4. High income

The order is meaningful; the labels do not specify how large the income differences are.

Natural Earth v5.1.2 · public domain · Historical lab classifications, not current income groups; observation year unspecified.
`null` means missing / not applicable (Antarctica), not a fifth income level.

set up a colormap

```
const levels = [
  "Low income",
  "Lower middle income",
  "Upper middle income",
  "High income"
];
const color = d3.scaleOrdinal()
  .domain(levels)
  .range(d3.schemeBlues[4])
  .unknown("#b8bec8");
```

`d3.scaleOrdinal()`

Creates a scale that maps discrete categories to output values. `domain(levels)` lists the income groups; `range(...)` assigns a color to each group in the same order.

D3 color schemes

D3 provides ready-made categorical, sequential, and diverging palettes. `d3.schemeBlues[4]` is an array of **four blues, from light to dark**—a sequential palette suited to ordered income levels.

`unknown("#b8bec8")`

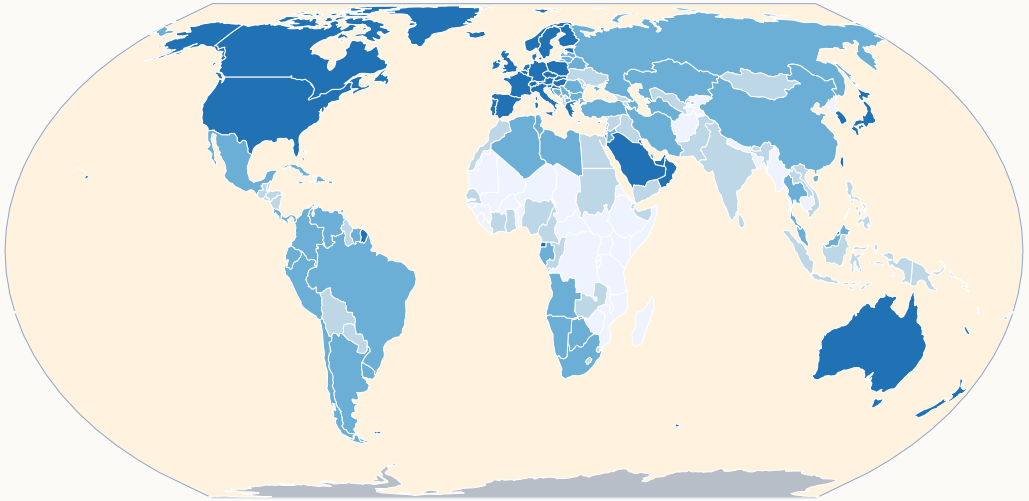
Uses gray for missing or unrecognized groups.

D3 ordinal scales · D3 sequential color schemes · The supplied D3 bundle includes these palettes.

Each country’s income group determines its fill color

```
// Draw all countries
svg.selectAll("path.country")
  .data(world.features)
  .enter().append("path")
  .attr("class", "country")
  .attr("d", d => path(d))
  .attr("fill", d =>
    color(d.properties.income_group))
  .attr("stroke", "white")
  .attr("stroke-width", 0.5);
```

`color` is the colormap from the previous slide: `income_group` → `color(...)` → `fill`.



How the fill callback works

D3 calls `d =>` `color(d.properties.income_group)` for each country path. `d` is one element of `world.features`: the Feature bound to that path. The returned color becomes its `fill`.

Read income_map.js ↗ · Natural Earth v5.1.2 · public domain · Historical lab classifications; observation year unspecified.

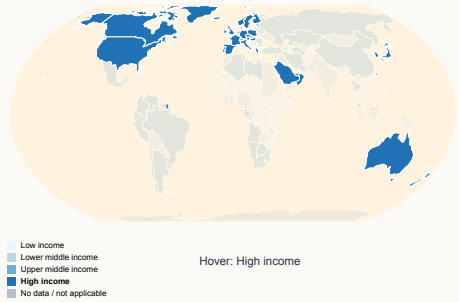
Use AI to extend your working map

Prompt 1 · Add a legend

“Extend income_map.js with a legend. Reuse its color scale: four income groups plus gray. Keep the map code; explain the changes. Save as legend_map.js.”

Prompt 2 · Add mouse events

“Extend that legend version with D3 mouseenter/mouseleave: highlight matching countries, dim others, then reset. Handle gray; explain (event, d). Save as interactive_map.js.”



Switch in Worldmap.html

Map script `src`:
`legend_map.js` → `interactive_map.js`.
Keep D3; load **one map script**.

Create first, compare afterward

Finish and test your AI-assisted scripts first.
Then check `legend_map_sample_code.js` and `interactive_map_sample_code.js`.
Do not use the references as your solution.

Run Worldmap.html with Live Server; test all five legend groups, mouseleave reset, and Console errors.
Target shown: hover High income. Map: Natural Earth v5.1.2 · public domain · Historical groups; observation year unspecified.

Appendix A · Add a legend

Legend rows are data-bound groups

```
const legendLevels = levels.concat([
  "No data / not applicable"
]);
const legend = svg.append("g")
  .attr("transform",
"translate(30,510)");
const items = legend
  .selectAll("g.legend-item")
  .data(legendLevels)
  .enter().append("g")
  .attr("class", "legend-item")
  .attr("transform", (d, i) =>
    "translate(0," + i * 25 + ")");
```

What is an SVG <g>?

A `<g>` groups SVG elements; it draws no shape by itself. Its `transform` moves its children together. The outer group places the legend at `(30, 510)`.

One category → one legend row

`data(legendLevels)` binds the category strings. `enter().append("g")` creates one row group per category: four income groups plus one missing-data row.

Here `d` is a category string and `i` is its index. `i * 25` spaces the rows vertically. Next, each row receives a rectangle and a text label.

Each legend row gets a swatch and a label

```
items.append("rect")
  .attr("width", 18)
  .attr("height", 18)
  .attr("fill", d => color(d));
items.append("text")
  .attr("x", 28).attr("y", 14)
  .text(d => d);
```

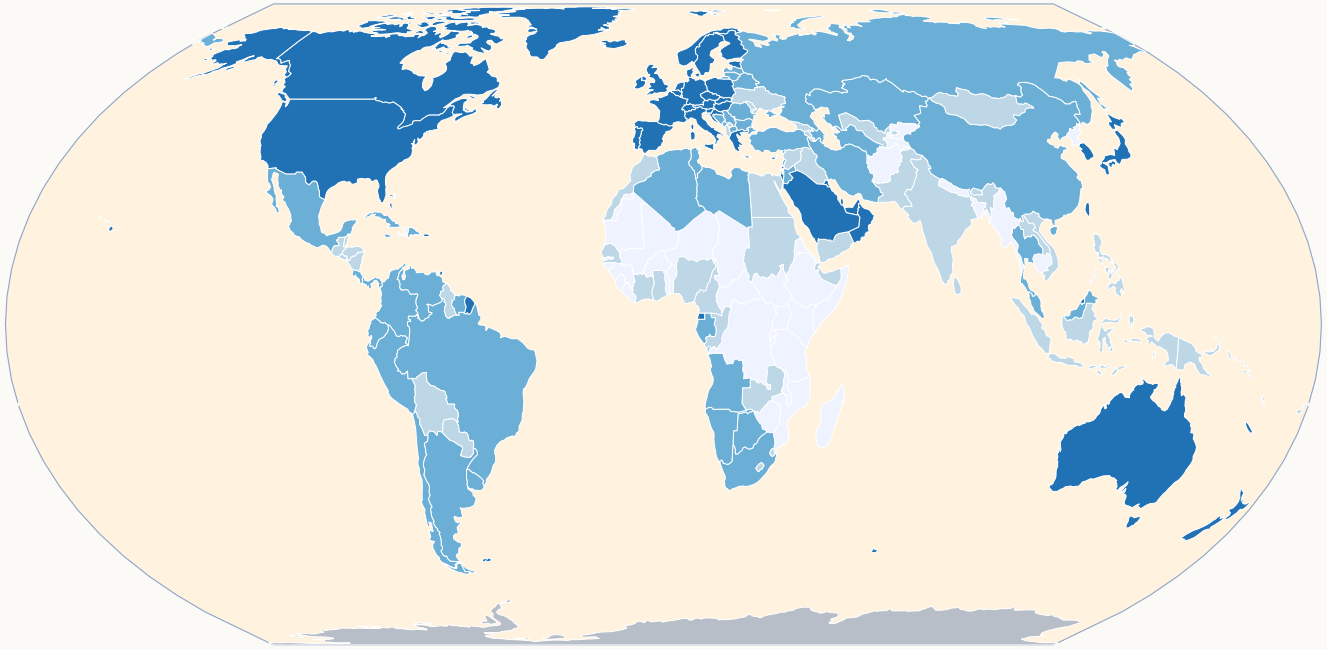
The child elements inherit their parent row’s bound category.

The **same color function** fills the countries and the legend swatches.

The label starts 28 units to the right; its baseline is 14 units below the row’s origin.

Check: every blue and the gray on the map must have the same meaning in the legend.

The finished map includes its interpretation



Read legend_map_sample_code.js ↗ · To run: set Worldmap.html's map script src to this file.

Natural Earth v5.1.2 · public domain · legacy income groups; observation year unspecified

Appendix B · Make the map interactive

A D3 event callback can update existing countries

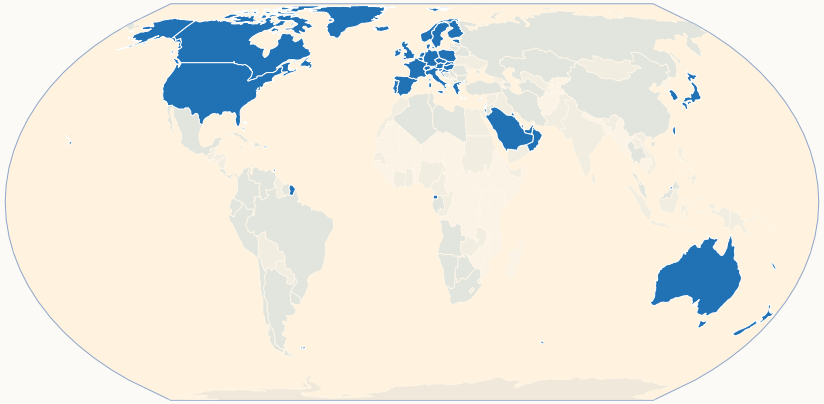
An **event listener** runs a function when an interaction occurs. In D3, the callback receives the browser event and the element’s bound datum.

```
function highlight(level) {
  countries.attr("opacity", d =>
    level === null || d.properties.income_group === level
      ? 1 : 0.2);
}
items.on("mouseenter", (event, d) => highlight(d))
  .on("mouseleave", () => highlight(null));
```

Here `d` is the legend category. This excerpt handles the four income groups; the complete example also handles the gray category.

Keyboard focus can reveal the same groups

```
items.attr("tabindex", 0)
  .on("focus", (event, d) => highlight(d))
  .on("blur", () => highlight(null));
```



Hover a legend row, then press Tab to focus the same row. The same countries should remain highlighted.

Optional extension: add these keyboard listeners to interactive_map_sample_code.js. The supplied script demonstrates mouseenter and mouseleave.

Read interactive_map_sample_code.js ↗ · To run, set Worldmap.html's map script src to this file.

Natural Earth v5.1.2 · public domain · legacy income groups; observation year unspecified

Appendix C · Join a CSV table

A CSV can store attributes separately from geometry

```
code,name,income_group
CHN,China,Upper middle income
IND,India,Lower middle income
ATA,Antarctica,
```

The **geometry file** stores shapes and stable codes. The **CSV table** stores attributes with the same codes.

`d3.csv()` returns an array of row objects. These three displayed columns arrive as strings.

We use Natural Earth's `code` field on both sides. Names are labels, not the join key.

The full CSV also preserves `source_income_group` for traceability.

Load the geometry, then load the table

```
d3.json("data/world.geojson")
  .then(world => {
    return d3.csv("data/income.csv").then(rows => {
      // Both world and rows are available here.
      // Match the rows, then call drawMap(world).
    });
  })
  .catch(showError);
```

The inner callback can use `world` from the outer callback. Returning the CSV Promise lets the final `catch` handle errors from either request.

D3 fetch: csv

find locates the row with the same country code

```
world.features.forEach(feature => {
  const row = rows.find(
    d => d.code === feature.properties.code
  );
  feature.properties.income_group =
    row ? row.income_group || null : null;
});
drawMap(world);
```

`find` returns the first matching row, or `undefined`. A blank field or absent row becomes `null`, which the color scale displays as gray.

The drawing function can now use the same `properties.income_group` as income_map.js.

Matching and missing values need different checks

Check	Supplied files	Meaning
Features / CSV rows	177 / 177	Each file uses unique codes
Unmatched features	0	Every feature has a table row
Unmatched rows	0	Every row has a feature
Missing / not applicable	1	Antarctica matches, but has no applicable income value

Read legend_map_sample_code.js ↗ · legend_map_sample_code.js loads and joins the CSV.

Extension: change one country code in a copy of the CSV. Predict both unmatched counts before reloading.

Appendix D · Complete reference

Complete reference · HTML setup

Set Worldmap.html to load `interactive_map_sample_code.js`, then open it with Live Server. The next three pages show the complete logic. The reference file adds teaching comments and expanded formatting.

```
<!-- Worldmap.html: keep the local D3 script -->
<script src="lib/d3.v7.9.0.min.js"></script>
<!-- Replace outline_map.js with this ONE script -->
<script src="interactive_map_sample_code.js" defer></script>
<svg id="map" viewBox="0 0 1000 980"></svg>
```

The data is `data/world-income.geojson`. The map uses the upper area; the legend starts at `y = 510`.

Read interactive_map_sample_code.js ↗ · Download all examples ↓

Complete reference · 1/3 · Load and project

```
const svg = d3.select("#map");
const levels = ["Low income", "Lower middle income",
  "Upper middle income", "High income"];
const missing = "No data / not applicable";
const color = d3.scaleOrdinal().domain(levels)
  .range(d3.schemeBlues[4]).unknown("#b8bec8");
d3.json("data/world-income.geojson")
  .then(drawMap).catch(error => console.error(error));
function drawMap(world) {
  const projection = d3.geoEqualEarth()
    .fitExtent([[20, 20], [980, 470]], world);
  const path = d3.geoPath(projection);
  svg.append("path").attr("d", path({type: "Sphere"}))
    .attr("fill", "#fff3df");
```

Combine parts 1–3 in order as `interactive_map_sample_code.js`. Read interactive_map_sample_code.js ↗

Complete reference · 2/3 · Draw countries and bind legend rows

```
  const countries = svg.selectAll("path.country")
    .data(world.features).enter().append("path")
    .attr("class", "country")
    .attr("d", d => path(d))
    .attr("fill", d => color(d.properties.income_group))
    .attr("stroke", "white").attr("stroke-width", 0.5);
  const legend = svg.append("g")
    .attr("transform", "translate(30,510)");
  const items = legend.selectAll("g")
    .data(levels.concat([missing])).enter().append("g")
    .attr("transform", (d, i) => "translate(0," + i * 25 + ")");
```

Combine parts 1–3 in order as `interactive_map_sample_code.js`. Read interactive_map_sample_code.js ↗

Complete reference · 3/3 · Legend and mouse events

```
  items.append("rect").attr("width", 18).attr("height", 18)
    .attr("fill", d => color(d));
  items.append("text").attr("x", 28).attr("y", 14)
    .attr("font-size", 17).text(d => d);
  function highlight(level) {
    countries.attr("opacity", d => {
      const group = d.properties.income_group || missing;
      return level === null || group === level ? 1 : 0.2;
    });
  }
  items.on("mouseenter", (event, d) => highlight(d))
    .on("mouseleave", () => highlight(null));
}
```

Combine parts 1–3 in order as `interactive_map_sample_code.js`. Read interactive_map_sample_code.js ↗